

Nome e Cognome: Angelo Condori

Classe: 3b TLC

Titolo del gioco: Orbital Cleanup: La Sfida di MATATO

Data della consegna: 04/06/2026

Il videogioco realizzato è ambientato nello spazio profondo. L'obiettivo principale del giocatore è comandare una navicella spaziale per distruggere i meteoriti che cadono dall'alto. Il gioco si articola su tre livelli di difficoltà progressiva; al raggiungimento del terzo livello, fa la sua comparsa il boss finale, che ho chiamato "MATATO", che si muove orizzontalmente lanciando raffiche di meteoriti distruttivi. Per vincere, il giocatore deve uccidere il boss e azzerarli i suoi 50 punti di vita. Si perde istantaneamente se la navicella entra in collisione con un asteroide, con il boss o con uno dei suoi colpi.

Il gioco è destinato ai compagni di classe e agli appassionati di queste dinamiche, ma anche ai bambini adulti e anziani che si vogliono divertire. Rispetto ai classici tutorial online, l'originalità del prodotto risiede nell'aggiunta di un sistema di "Sparo Triplo" che si sblocca solo dopo aver raccolto 10 monete d'oro e in un pattern di attacco del boss basato su raffiche di colpi consecutivi molto veloci.

Mi sono fatto aiutare dall'intelligenza artificiale per il codice del gioco, per perfezionare tutto il videogioco e anche farmi correggere i vari errori grammaticali nella relazione.

3.1 Regole del gioco

- La navicella si muove solo orizzontalmente sul fondo dello schermo.
- Ogni meteorite standard distrutto assegna 15 punti. La raccolta di una moneta assegna 10 punti.
- Al raggiungimento di 10 monete, l'arma del giocatore si potenzia automaticamente attivando il triplo sparo dei proiettili.
- Il passaggio di livello avviene in base al punteggio: il Livello 2 si attiva a 100 punti, il Livello 3 cioè la fase del boss arriva a 300 punti.
- Il giocatore ha una sola vita, infatti se viene preso da un meteorite, finisce il gioco e si determina la sconfitta.

3.2 Struttura del programma

Il programma è stato sviluppato in Python utilizzando i seguenti moduli:

- **pygame**: impiegato per la gestione della finestra, del game loop, del rendering grafico, delle collisioni e del comparto audio.
- **random**: essenziale per generare le coordinate casuali di generazione dei meteoriti e delle monete, nonché per variare la velocità dei flussi.
- **sys**: utilizzato per la chiusura pulita dell'applicazione.

schemino a blocchi:

[Avvio Programma]

|



[Mostra Schermata Menu] —(Pressione tasto X)—▶ [Inizializzazione Entità]

|



[Ciclo di Gioco (60 FPS)]

|— Lettura Input

|— Aggiornamento Posizioni & Neve

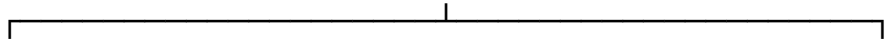
|— Controllo Collisioni

|— Rendering Grafico

|

(Verifica Condizioni di Fine)

|



[Collisione = True]

|



[Schermata Game Over]



[Vita Boss $\leq$ 0]

|



[Schermata Vittoria]

3.3 Strutture dati importanti

Il programma sfrutta intensamente la programmazione a oggetti mediante l'estensione della classe `pygame.sprite`.

- **Giocatore:** Rappresenta l'entità controllata dall'utente. Gestisce la posizione, la velocità di movimento, il contatore delle monete e il metodo `spara()`.
- **Ostacolo:** Gestisce i singoli meteoriti. Calcola una velocità casuale iniziale incrementata in base al livello corrente del gioco.
- **BossMatato:** Gestisce l'antagonista finale, il suo movimento alternato da sinistra a destra e il calcolo dei frame per la gestione dei colpi.
- **MeteoriteBoss:** Sottoclasse specifica per i proiettili del boss, dotata di una velocità verticale fissa e molto elevata
- **lista_neve:** Una struttura dati nativa che memorizza le coordinate [X, Y] e la velocità di 100 particelle per generare l'effetto visivo della neve sullo sfondo.

4.1 Come ho lavorato

Lo sviluppo è seguito in modo lineare. Sono partito dalla stesura dell'idea e dei requisiti minimi, per poi passare alla configurazione della finestra e del movimento base. La parte che ha richiesto più tempo e che ha presentato le maggiori difficoltà è stata la gestione del tempo per i colpi del boss: coordinare i frame per fare in modo che l'attacco non fosse un flusso continuo ma una raffica distinta ha richiesto diversi tentativi e riscritture logiche.

4.2 Come ho usato l'AI

Per il completamento del progetto ho utilizzato l'assistente Gemini. E' stato sfruttato in modo interattivo per superare i limiti delle mie attuali competenze di programmazione, in particolare sulla gestione della programmazione a oggetti e dei vettori di particelle.

I prompt principali inviati all'AI sono stati:

1. *"Metti nel codice che tutti i meteoriti (anche quelli che lancia il boss) devono andare più velocemente così da essere più difficile vincere."*
2. *"Fai una schermata di inizio del gioco dove spieghi i comandi, dici 'clicca x per iniziare l'avventura!' e assicurati che se premo un tasto sbagliato il gioco non crashi."*
3. *"Fai sì che il boss lanci due o tre palle consecutivamente a raffica."*

Modifiche e comprensione del codice: Non ho copiato passivamente il codice generato. Ho dovuto adattare i nomi dei file delle immagini a quelli realmente presenti nella mia cartella di lavoro (es. `navicella.png`, `asteroide.png`). Inoltre, ho analizzato riga per riga il funzionamento del ciclo `for` che aggiorna la coordinata `fiocco[1]` della neve, comprendendo e capendo bene come il reset della coordinata Y a valori negativi permetta il riciclo infinito dei punti sullo schermo senza saturare la memoria

5.1 Casi di test

La verifica delle funzionalità è stata eseguita sistematicamente durante le sessioni di compilazione:

Test	Cosa ho fatto	Cosa mi aspettavo	Cosa è successo
1	Pressione di tasti casuali (es. 'A', 'M') nel menu d'avvio.	Il menù deve ignorare l'input senza bloccarsi.	Funziona: l'input asincrono viene scartato dal blocco pass e si ferma tutto.
2	Pressione del tasto 'X' nel menu d'avvio.	Chiusura del menu e avvio immediato della partita.	Funziona correttamente.
3	Raccolta di 10 monete d'oro durante la partita.	Attivazione della stringa a schermo e dello sparo a tre vie.	Funziona: la navicella spara a ventaglio.
4	Collisione della navicella con un meteorite del boss.	Interruzione del loop e passaggio alla schermata di Game Over.	Funziona.

5.2 Bug e problemi

- **Risolto (Crash audio):** Inizialmente il programma andava in crash sui computer del laboratorio privi di cuffie o schede audio connesse, restituendo l'errore. Il problema è stato superato isolando l'inizializzazione del mixer all'interno di un blocco try/except. Se l'hardware audio è assente, il programma cattura l'eccezione e attiva automaticamente una modalità silenziosa, evitando il crash dell'applicazione.
- **Risolto (Trasparenze immagini):** L'uso iniziale del metodo `set_colorkey(BIANCO)` creava un fastidioso effetto sgranato attorno agli sprite se i bordi non erano perfettamente uniformi. Il bug è stato corretto convertendo le risorse grafiche in veri file .png con canale alpha integrato e modificando il caricamento nel codice con la funzione `.convert_alpha()`

6. Conclusioni e riflessione personale

Questo progetto mi ha permesso di capire a fondo la struttura di un'applicazione multimediale in tempo reale e l'importanza del bilanciamento della difficoltà. Sento proprio che mi ha insegnato molte cose del mondo informatico e che ho imparato nuovi concetti. La gestione del tempo tramite il ticking del clock a 60Hz è stata la nozione teorica più interessante che ho appreso. La parte più complessa è stata l'implementazione della logica dei colpi a raffica del boss, ma l'analisi delle variabili di stato temporanee consigliate dall'AI mi ha aiutato a superare l'ostacolo. Ritengo di aver utilizzato l'intelligenza artificiale come un vero e proprio tutor di programmazione: invece di farmi scrivere il gioco da zero senza capire nulla, ho decostruito il codice chiedendo spiegazioni sui singoli moduli e sulle collisioni tra gruppi di sprite. Se dovessi rifare il progetto, implementerei un sistema di salvataggio locale dei punteggi più alti su un file di testo esterno.

7. Allegati

